

Федеральное государственное бюджетное образовательное учреждение
высшего образования «Сибирский государственный университет телекоммуникаций
и информатики»

институт ИВТ

Кафедра вычислительных систем

Расчетно-графическая работа
на тему «Сжатие данных»
Вариант 4.1 «Алгоритм Шеннона-Фано (Shannon-Fano)»

Выполнил:
студент гр. ИВ-521
Шматок И.А

Проверил:
старший преподаватель кафедры ВС
Ревун А.Л.

Тема расчетно-графической работы

Сжатие данных.

Задание на расчетно-графическую работу

Реализовать программу `sfcompress` сжатия текстовых файлов на английском языке алгоритмом Шеннона-Фано. Сжатие осуществляется с аргументом командной строки `-c` (`compress`), а распаковка – с аргументом `-d` (`decompress`). Опция `-o` указывает имя выходного файла. Например:

```
$ sfcompress -c -o file.sfc file.txt #сжатие file.txt в file.sfc
$ sfcompress -d -o file1.txt file.sfc # распаковка file.sfc в file1.txt
```

Критерии оценки

- Оценка «хорошо»: реализован алгоритм сжатия, не обеспечено компактное расположение сжатого текста в выходном файле – код каждого символа занимает целое число байт.
- Оценка «отлично»: закодированный текст компактно упаковывается в файл без учета байтовых границ (с помощью битового массива, как описано в общей информации к разделу).

Указание к выполнению задания Алгоритм Шеннона-Фано использует коды переменной длины. Часто встречающийся символ кодируется кодом меньшей длины, редко встречающийся – кодом большей длины. Например, исходный текст:

«ббббмдддууууууууу». Частоты встречающихся в исходном тексте символов: б – 4, м – 1, 'д' – 3, 'у' – 6.

Основные этапы алгоритма Шеннона-Фано:

1. Список X символов упорядочивается по убыванию частоты встречаемости.
2. Список X делится на две части X1 и X2 так, чтобы разница сумм частот обеих частей ($\Sigma 1$ и $\Sigma 2$) была минимальна, например $\Sigma 1 = 6$ (симв. 'у'), $\Sigma 2 = 8$ (симв. 'б', 'м', 'д').
3. Кодовой комбинации каждого символа дописывается 1 (X1) или 0 (X2).
4. Если X1 содержит один символ, код этого символа построен, перейти к шагу 5.
5. Иначе – рекурсивно перейти к шагу 2, считая $X = X1$. Если X2 содержит один символ, код этого символа построен – завершить данный шаг рекурсии (на предыдущих шагах рекурсии могут быть необработанные данные). Если символов больше одного – рекурсивно перейти на шаг 2, считая, что $X = X2$.

Для распаковки сжатых данных необходимо знать все использованные коды, поэтому требуется сохранять полученную таблицу соответствия «символкод» в сжатом файле.

Текст «ббббмдддууууууууу» будет закодирован в виде двоичной последовательности:
01|01|01|01|000|001|001|001|1|1|1|1|1|1|1|1

Анализ задачи

1. types.h

- 1.1. Структура `CodeEntry` – запись таблицы кодов Шеннона-Фано + через атрибут для упаковки компилятором без выравнивания (т. е. будет 6 байт на структуру)

Переменная	Размер	Назначение
symbol	1байт	Код символа в ASCII
code_len	1байт	Длина кода в битах (1-32)
code_val	4байта	Значение

- 1.2. Структура `SymFreq` – для сортировки по частоте

Переменная	Размер	Назначение
symbol	1байт	Символ

freq	8байт	Частота
------	-------	---------

1.3. Структура BitBuffer – динамический бит. буфер

Переменная	Тип	Назначение
data	uint8_t*	Указатель на массив байт
capacity	size_t	Емкость буфера

2. shennon.h и shennon.c

2.1. В заголовочном файле только `int build_shannon_table(const uint8_t *data, size_t data_len, CodeEntry *table);`

2.2. В файле с функциями:

2.2.1. Поиск точки для разбиения – `find_split(syms, start, end)`

Найти такой индекс, при котором разница между суммами частот левой `[start, split)` и правой `[split, end)` частей минимальна.

`total ← сумма syms[i].freq для i = start..end-1`

для i от start до end-2:

`left_sum += syms[i].freq`

`right_sum = total - left_sum`

`diff = |left_sum - right_sum|`

если `diff < min_diff`:

`min_diff = diff`

`best_split = i + 1`

вернуть `best_split`

2.2.2. Рекурсивно строим код – `sf_build(syms, start, end, code, depth, table, idx)`

Рекурсивно присвоить коды символам по алгоритму Шеннона-Фано.

`//если 1 символ в диапазоне`

если `start == end - 1`:

`table[*idx].symbol = syms[start].symbol`

`table[*idx].code_len = depth`

`table[*idx].code_val = code`

`*idx = *idx + 1`

вернуть

`//рекурсивно: разбиваем`

`split = find_split(syms, start, end)`

`//в левую ветвь: добавляем бит 0`

`sf_build(syms, start, split, code << 1, depth + 1, table, idx)`

`//в правую ветвь: добавляем бит 1`

`sf_build(syms, split, end, (code << 1) | 1, depth + 1, table, idx)`

2.2.3. Основная функция и точка входа – `build_shannon_table (data, data_len, table)`

`//1. подсчёт частот`

`freq[256] = {0}`

для i = 0 до `data_len-1`:

`freq[data[i]].symbol = data[i]`

`freq[data[i]].freq++`

`// 2. фильтрация и копирование ненулевых частот`

`sorted[] = пустой массив`

для s = 0 до 255:

если `freq[s].freq > 0`:

добавить `freq[s]` в `sorted`

```
// 3. сортировка по убыванию частоты
qsort(sorted, cmp_freq_desc)
```

```
// 4. построение таблицы кодов
table_idx = 0
sf_build(sorted, 0, len(sorted), 0, 0, table, &table_idx)
```

вернуть количество символов

2.2.4. Пример работы: строка: "ббббмдддууууу", частоты у=6, б=4, д=3, м=1.

1 – сортировка по частотам: индекс/символ/частота: 0/у/6; 1/б/4; 2/д/3; 3/м/1.

2 – первое разбиение `find_split`

`total` = 6+4+3+1 = 14

`split` = 1: `left` = 6, `right` = 8, `diff` = 2 (MINIMUM)

`split` = 2: `left` = 10, `right` = 4, `diff` = 6

`split` = 3: `left` = 13, `right` = 1, `diff` = 12

Тогда в левой части будет [0, 1): {у} – код 1

В правой части [1,4): {б, д, м} – с префиксом 0

3 – Рекурсия для правой части {б, д, м}:

Частоты – 4, 3, 1

`total` = 8

`split` = 2: `left`=4, `right` = 4, `diff` = 0 (MINIMUM)

Тогда в левой части будет: {б} – код 01

В правой части: {д, м} – с префиксом 00

4 – Рекурсия для правой части {д, м}:

Частоты – 3, 1

`total` = 4

`split` = 1: `left` = 3, `right` = 1, `diff` = 2 (MINIMUM)

Тогда в левой части будет: {д} – код 001

В правой части: {м} – с кодом 000

И мы получили длинные бинарные коды для редких символов, а короткие для частых.

б	б	б	б	м	д	д	д	у	у	у	у	у	у
01	01	01	01	000	001	001	001	1	1	1	1	1	1

01010101	00000100	10011111	11000000
0 байт	1 байт	2 байт	3 байт

Исходный размер 14 байт

Сжатый размер = 4 байт

К сжатия = 14/4 = 3.5

Но в моей программе нужно учитывать заголовок, если смотреть размер файла полностью.

3. `bitops.c`

3.1. Битовый массив как структура:

Данные хранятся в массиве байтов `uint8_t[]`, но представляют собой последовательность

битов с глобальной нумерацией.
Так выглядит адресация:

Глобал. бит. индекс	0	1	2	3	4	5	6	7
Байт 0	b7	b6	b5	b4	b3	b2	b1	b0

	0	1	2	3	4	5	6	7
Байт 1	b7	b6	b5	b4	b3	b2	b1	b0

Порядок битов идет по принципу MSB, то есть старший бит идет первым. Это соответствует визуальному представлению двоичных чисел (в xhd -b нормально отобразится).

3.2. Функция setbits(arr, offset, val, len) – запись битового поля

Записать len младших битов значения val в массив arr, начиная с глобального смещения offset (между байтами)

для i от 0 до len-1:

//извлекаем i-й бит из val (начиная со старшего)

bit = (val >> (len - 1 - i)) & 1

//вычисляем позицию в массиве байтов

byte_idx = (offset + i) / 8

bit_shift = 7 - ((offset + i) % 8) //реализация MSB (инвертирование)

//очищаем целевой бит и устанавливаем новое значение

arr[byte_idx] = (arr[byte_idx] & ~(1 << bit_shift)) | (bit << bit_shift)

3.3. Функция getbits(arr, offset, len) – чтение битового поля

Прочитать len бит из массива arr, начиная с offset, и вернуть их как целое число

res = 0

для i от 0 до len-1:

byte_idx = (offset + i) / 8

bit_shift = 7 - ((offset + i) % 8)

bit = (arr[byte_idx] >> bit_shift) & 1

res = (res << 1) | bit //сдвигаем результат и добавляем новый бит

вернуть res

4. archive.c

4.1. Формат выходного файла:

Заголовок:

а) [num_sys: uint32_t] 4 байта (кол-во уник символов)

б) [orig_len: uint64_t] 8 байт (исходный размер файла)

в) структура CodeEntry x num_sys 6 байт

symbol: uint8_t 1байт

code_len: uint8_t 1байт

code_val: uint32_t 4байта

Битовый поток:

Упакованные коды символов, в последнем байте возможно дополнение нулями

Размер заголовка: 4 + 8 + 6*num_sys байт

4.2. Функция compress(inpath, outpath):

Читает, сжимает входные данные и записывает в файл.

// 1. чтение исходника

data = прочитать весь файл *in_path* в память

file_size = размер данных

// 2. построение таблицы кодов Шеннона–Фано

table = пустой массив *CodeEntry*[256]

num_syms = *build_shannon_table*(*data*, *file_size*, *table*)

// 3. создание lookip-таблицы для быстрого поиска

lookup[256] = {NULL}

для *i* от 0 до *num_syms*-1:

lookup[*table*[*i*].symbol] = &*table*[*i*]

// 4. расчёт общего количества бит в сжатом потоке

total_bits = 0

для каждого байта *b* в *data*:

total_bits = *total_bits* + *lookup*[*b*]->*code_len*

// 5. инициализация битового буфера

buf = новый *BitBuffer* с ёмкостью (*total_bits* + 7) / 8 + 16

bit_offset = 0

// 6. кодирование сообщения

для каждого байта *b* в *data*:

entry = *lookup*[*b*]

buf *ensure*(&*buf*, (*bit_offset* + *entry*->*code_len* + 7) / 8)

setbits(*buf.data*, *bit_offset*, *entry*->*code_val*, *entry*->*code_len*)

bit_offset = *bit_offset* + *entry*->*code_len*

// 7. запись сжатого файла

fout = открыть *out_path* для бинарной записи

записать *num_syms* (4 байта)

записать *orig_len* (8 байт)

записать *table*[0..*num_syms*-1] (по 6 байт каждая запись)

записать *buf.data*[0..(*bit_offset*+7)/8 - 1] (битовый поток)

закреть *fout*

// 8. вывод статистики

out_size = размер заголовка + (*bit_offset* + 7) / 8

k = *file_size* / *out_size*

вывести "Сжатие: {*file_size*} -> {*out_size*} байт (*k*= {*k*})"

4.3. Функция *decompress*(*in_path*, *out_path*)

Читает, декодирует и записывает в файл.

// 1. чтение заголовка

fin = открыть *in_path* для бинарного чтения

прочитать *num_syms* (uint32_t)

прочитать *orig_len* (uint64_t)

выделить *table* = *malloc*(*num_syms* × *sizeof*(*CodeEntry*))

прочитать *table*[0..*num_syms*-1]

// 2. чтение битового потока

data_start = текущая позиция в файле

bitstream_size = размер файла - *data_start*

```
выделить bitstream = malloc(bitstream_size)
прочитать bitstream
```

```
// 3. декодирование
```

```
fout = открыть out_path для записи
```

```
acc = 0 // аккумулятор битов (для разделения битов)
```

```
acc_len = 0 // длина накопленного кода
```

```
decoded = 0 // счётчик восстановленных символов
```

```
total_bits = bitstream_size × 8
```

```
пока decoded < orig_len И bit_offset < total_bits:
```

```
    bit = getbits(bitstream, bit_offset, 1) //читаем следующий бит из потока
```

```
    acc = (acc << 1) | bit //добавляем бит в аккумулятор
```

```
    acc_len = acc_len + 1
```

```
    bit_offset = bit_offset + 1
```

```
//поиск совпадения в таблице кодов
```

```
для i от 0 до num_syms-1:
```

```
    если table[i].code_len == acc_len И table[i].code_val == acc:
```

```
        записать table[i].symbol в fout
```

```
        decoded = decoded + 1
```

```
        acc = 0 // сброс аккумулятора
```

```
        acc_len = 0
```

```
        прервать цикл поиска
```

```
закрыть fout
```

```
вывести "Распаковка: {decoded} символов восстановлено"
```

5. Точка входа в программу – main.c

5.1. Моя программа sfcompress имеет 2 типа работы через флаги при запуске: -c режим сжатия, -d режим распаковки, -o <out_file> имя выходного файла, <input_file> входной файл.

Примеры запуска:

а) сжатие: ./sfcompress -c -o archiv.sfc test.txt

б) распаковка: ./sfcompress -d -o detest.txt archiv.sfc

5.2. Псевдокод:

```
инициализация: c_flag=0, d_flag=0, out_file=NULL
```

```
// парс аргументов
```

```
пока (opt = getopt(argc, argv, "cdo:")) != -1:
```

```
    по opt:
```

```
        'c': c_flag = 1
```

```
        'd': d_flag = 1
```

```
        'o': out_file = optarg
```

```
    иначе: ошибка = return 1;
```

```
// проверка вход файла
```

```
если optind >= argc:
```

```
    вывести "Ошибка: не указан входной файл" = return 1
```

```
in_file ← argv[optind]
```

```
//проверка флагов
```

```
если out_file == NULL:
```

```
    вывести "Ошибка: необходимо указать -o" = return 1
```

```
если не (c_flag ИЛИ d_flag):
```

```
вывести "Ошибка: укажите -с или -d" = return 1
если c_flag И d_flag:
вывести "Ошибка: нельзя указать оба режима" = return 1
```

```
// выполнение функций
если c_flag: compress(in_file, out_file)
иначе: decompress(in_file, out_file)
```

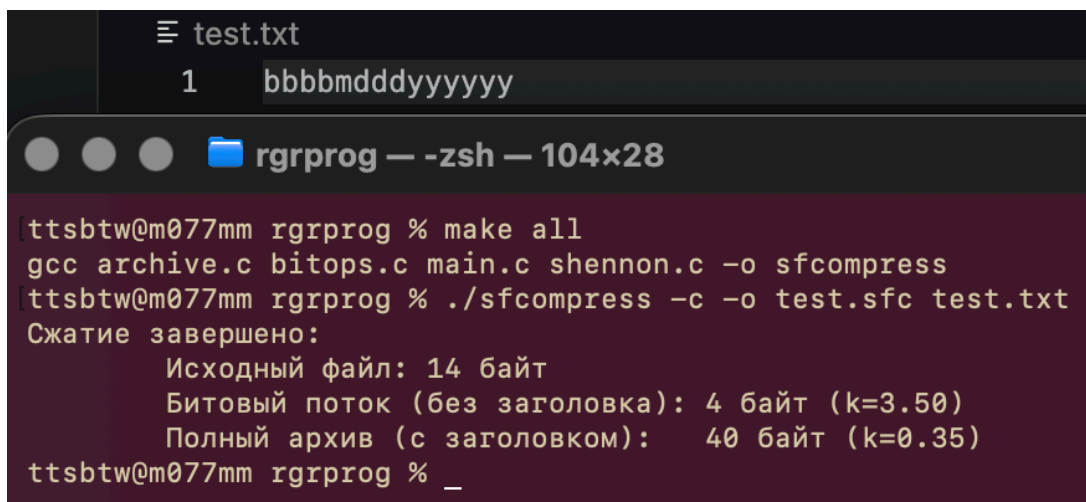
```
вернуть return 0
```

- 5.3. Использовал библиотеку getopt.h для работы с функциями opt. getopt() – последовательно перебирает переданные параметры в программу. Для флагов -с и -d нет никаких доп. аргументов. А для -о следующим идет название выходного файла, поэтому ставлю двоеточие после него. Записываю его в глобальную переменную optarg. Индекс optind указывает на первый аргумент после обработанных позиций.

Тестовые данные

1 тест – из разобранного в пункте 2.2.4.

Строка на вход “bbbbmdddyyyyy”. $b = 4$, $m = 1$, $d = 3$, $y = 6$. $\Rightarrow 4+1+3+6 = 14$ байт



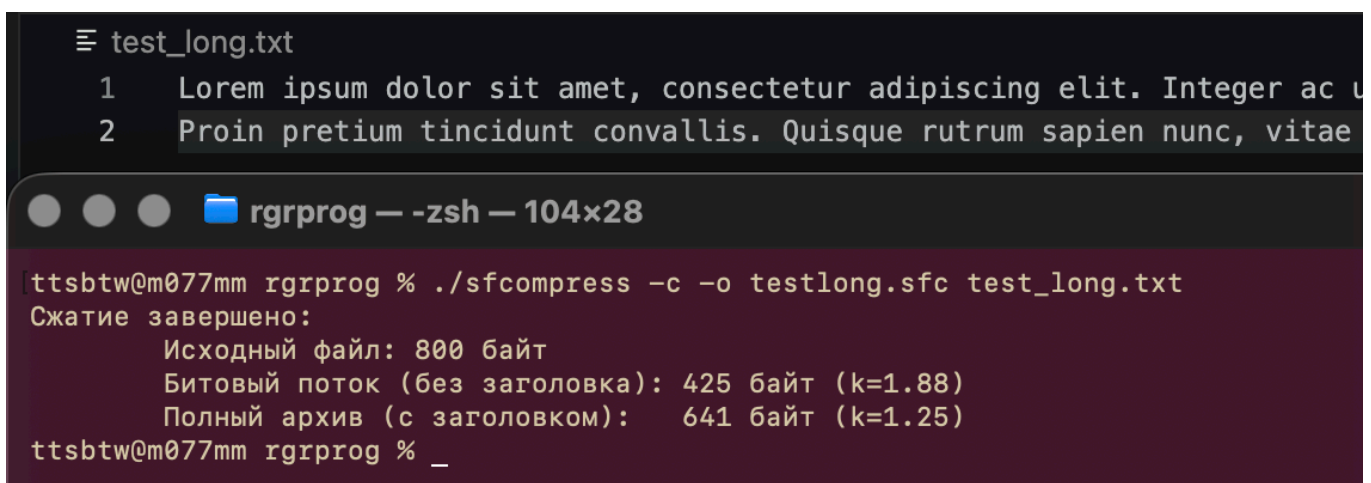
```
test.txt
1 bbbbmdddyyyyy

rgrprog — -zsh — 104x28

ttsbtw@m077mm rgrprog % make all
gcc archive.c bitops.c main.c shennon.c -o sfcompress
ttsbtw@m077mm rgrprog % ./sfcompress -c -o test.sfc test.txt
Сжатие завершено:
    Исходный файл: 14 байт
    Битовый поток (без заголовка): 4 байт (k=3.50)
    Полный архив (с заголовком): 40 байт (k=0.35)
ttsbtw@m077mm rgrprog % _
```

Что и требовалось из теории, битовый поток занял 4 байт.

2 тест – длинный текст по типу Lorem Ipsum на 800 символов (800 байт).



```
test_long.txt
1 Lorem ipsum dolor sit amet, consectetur adipiscing elit. Integer ac u
2 Proin pretium tincidunt convallis. Quisque rutrum sapien nunc, vitae

rgrprog — -zsh — 104x28

ttsbtw@m077mm rgrprog % ./sfcompress -c -o testlong.sfc test_long.txt
Сжатие завершено:
    Исходный файл: 800 байт
    Битовый поток (без заголовка): 425 байт (k=1.88)
    Полный архив (с заголовком): 641 байт (k=1.25)
ttsbtw@m077mm rgrprog % _
```

Здесь уже повторяющиеся символы не видны вооруженным глазом, но даже при этом программа кодируется даже с заголовком с >1 коэффициентом.

3 тест – повторяющиеся А 1000 раз (1000 байт).

```
repeat.txt
1 A
2 A
3 A

rgrprog — -zsh — 104x28

ttsbtw@m077mm rgrprog % ./sfcompress -c -o repeat.sfc repeat.txt
Сжатие завершено:
    Исходный файл: 1000 байт
    Битовый поток (без заголовка): 125 байт (k=8.00)
    Полный архив (с заголовком): 149 байт (k=6.71)
```

Коэффициент сжатия будет максимальным, потому что уникальных символов всего 1.

4 тест – пустой файл empty.txt. Программа не выполнится.

```
empty.txt
1 Generate code (%I), or select a language (%K M). Start

rgrprog — -zsh — 104x28

ttsbtw@m077mm rgrprog % ./sfcompress -c -o empty.sfc empty.txt
Пустой файл
```

5 тест – 1 символ V в one.txt. (Размер минимального заголовка 18байт)

```
one.txt
1 V

rgrprog — -zsh — 104x28

ttsbtw@m077mm rgrprog % ./sfcompress -c -o one.sfc one.txt
Сжатие завершено:
    Исходный файл: 1 байт
    Битовый поток (без заголовка): 1 байт (k=1.00)
    Полный архив (с заголовком): 19 байт (k=0.05)
```

6 тест – распаковка test.sfc (из 1 теста) в detest.txt

```
rgrprog — -zsh — 104x28

ttsbtw@m077mm rgrprog % ./sfcompress -d -o detest.txt test.sfc && cat detest.txt
Распаковка завершена: 14 символов восстановлено
bbbbmddduuuuu
```

7, 8, 9, 10 тест на невалидность ввода:

```
ttsbtw@m077mm rgrprog % ./sfcompress -o out.sfc in.txt
Ошибка: укажите -c или -d.
ttsbtw@m077mm rgrprog % ./sfcompress -c -d -o out.sfc in.txt
Ошибка: нельзя одновременно -c и -d.
ttsbtw@m077mm rgrprog % ./sfcompress -c -o in.txt
Ошибка: не указан входной файл.
ttsbtw@m077mm rgrprog % ./sfcompress -c -o out.sfc in.txt
Input file: No such file or directory
```


Листинг программы

types.h:

```
#ifndef TYPES_H
#define TYPES_H

#include <stdint.h>
#include <stddef.h>

//структура для хранения кода символа (1+1+4байта = 6байт)
typedef struct {
    uint8_t symbol;
    uint8_t code_len;
    uint32_t code_val;
} __attribute__((packed)) CodeEntry;

//вспомогательная структура для подсчёта частот
typedef struct {
    uint8_t symbol;
    uint64_t freq;
} SymFreq;

//динамический битовый буфер
typedef struct {
    uint8_t *data;
    size_t capacity; //в байтах
} BitBuffer;

#endif
```

shennon.h:

```
#ifndef SHENNON_H
#define SHENNON_H

#include "types.h"

//построение таблицы кодов Шеннона-Фано
int build_shannon_table(const uint8_t *data, size_t data_len, CodeEntry *table);

#endif
```

shennon.c:

```
#include "shennon.h"
#include <stdlib.h>
#include <string.h>

static int find_split(const SymFreq *syms, int start, int end) { //поиск точки разбиения
    uint64_t total = 0;
    for (int i = start; i < end; i++) total += syms[i].freq;

    uint64_t left_sum = 0;
    int best_split = start + 1;
    uint64_t min_diff = (uint64_t)-1;

    for (int i = start; i < end - 1; i++) {
        left_sum += syms[i].freq;
        uint64_t right_sum = total - left_sum;
        uint64_t diff = (left_sum > right_sum) ? (left_sum - right_sum) : (right_sum - left_sum);
        if (diff < min_diff) {
            min_diff = diff;
            best_split = i + 1;
        }
    }
    return best_split;
}

static void sf_build(SymFreq *syms, int start, int end, uint32_t code, int depth, CodeEntry *table, int *idx) { //рекурсия и постройка дерева
    if (start >= end) return;

    if (start == end - 1) {
```

```
        table[*idx].symbol = syms[start].symbol;
        table[*idx].code_len = depth;
        table[*idx].code_val = code;
        (*idx)++;
        return;
    }
    int split = find_split(syms, start, end);
    sf_build(syms, start, split, code << 1, depth + 1, table, idx);
    sf_build(syms, split, end, (code << 1) | 1, depth + 1, table, idx);
}

static int cmp_freq_desc(const void *a, const void *b) { //для qsort
    const SymFreq *fa = (const SymFreq *)a;
    const SymFreq *fb = (const SymFreq *)b;
    if (fb->freq != fa->freq) return (fb->freq > fa->freq) - (fa->freq > fb->freq); //основное сравнение по частоте
    return fa->symbol - fb->symbol; //вторич сравн по коду символа
}

int build_shannon_table(const uint8_t *data, size_t data_len, CodeEntry *table) { //построение таблицы
    SymFreq freq[256] = {0};
    for (size_t i = 0; i < data_len; i++) {
        freq[data[i]].symbol = data[i];
        freq[data[i]].freq++;
    }

    SymFreq sorted[256];
    int num_syms = 0;
    for (int i = 0; i < 256; i++) {
        if (freq[i].freq > 0) sorted[num_syms++] = freq[i];
    }
    qsort(sorted, num_syms, sizeof(SymFreq), cmp_freq_desc);

    int table_idx = 0;
    sf_build(sorted, 0, num_syms, 0, 0, table, &table_idx);
    return num_syms;
}
```

bitops.h:

```
#ifndef BITOPS_H
#define BITOPS_H
```

```
#include "types.h"
```

```
//управление битовым буфером
void buf_init(BitBuffer *buf, size_t capacity_bytes);
void buf_ensure(BitBuffer *buf, size_t needed_bytes);
void buf_free(BitBuffer *buf);
```

```
//побитовые операции (MSB-first)
void setbits(uint8_t *arr, size_t offset, uint32_t val, uint8_t len);
uint32_t getbits(const uint8_t *arr, size_t offset, uint8_t len);
```

```
#endif
```

bitops.c:

```
#include "bitops.h"
#include <stdlib.h>
#include <string.h>
#include <stdio.h>
```

```
void buf_init(BitBuffer *buf, size_t capacity_bytes) { //инициализация
    buf->capacity = capacity_bytes;
    buf->data = calloc(1, capacity_bytes);
    if (!buf->data) { perror("calloc"); exit(EXIT_FAILURE); }
}
```

```
void buf_ensure(BitBuffer *buf, size_t needed_bytes) { //расширение буфера
    if (needed_bytes > buf->capacity) {
        size_t new_cap = needed_bytes * 2;
        uint8_t *tmp = realloc(buf->data, new_cap);
        if (!tmp) { perror("realloc"); exit(EXIT_FAILURE); }
        memset(tmp + buf->capacity, 0, new_cap - buf->capacity);
    }
}
```

```

        buf->data = tmp;
        buf->capacity = new_cap;
    }
}

void buf_free(BitBuffer *buf) { //очистка буфера
    if (buf->data) free(buf->data);
    buf->data = NULL;
    buf->capacity = 0;
}

void setbits(uint8_t *arr, size_t offset, uint32_t val, uint8_t len) { //запись
бит поля
    for (int i = 0; i < len; i++) {
        int bit = (val >> (len - 1 - i)) & 1; //старший бит идёт первым
        size_t byte_idx = (offset + i) / 8; //позиция в массиве байтов
        int bit_shift = 7 - ((offset + i) % 8); //реализация мсб
        arr[byte_idx] = (arr[byte_idx] & ~(1 << bit_shift)) | (bit << bit_shift);
    }
    //очищаем и устанавливаем новое
}

uint32_t getbits(const uint8_t *arr, size_t offset, uint8_t len) { //чтение бит
поля
    uint32_t res = 0;
    for (int i = 0; i < len; i++) {
        size_t byte_idx = (offset + i) / 8;
        int bit_shift = 7 - ((offset + i) % 8);
        res = (res << 1) | ((arr[byte_idx] >> bit_shift) & 1); //сдвиг результата
        //и добавляем новый бит
    }
    return res;
}

```

archive.h:

```

#ifndef ARCHIVE_H
#define ARCHIVE_H

//сжатие и распаковка файлов
void compress(const char *in_path, const char *out_path);
void decompress(const char *in_path, const char *out_path);

#endif

```

archive.c:

```

#include "archive.h"
#include "bitops.h"
#include "shannon.h"
#include <stdio.h>
#include <stdlib.h>

void compress(const char *in_path, const char *out_path) { //сжатие
    FILE *fin = fopen(in_path, "rb");
    if (!fin) { perror("Input file"); return; }

    fseek(fin, 0, SEEK_END);
    long file_size = ftell(fin);
    if (file_size <= 0) { fprintf(stderr, "Пустой файл\n"); fclose(fin); return; }
    rewind(fin);

    uint8_t *data = malloc(file_size);
    fread(data, 1, file_size, fin);
    fclose(fin);

    CodeEntry table[256];
    int num_syms = build_shannon_table(data, file_size, table);

    CodeEntry *lookup[256] = {NULL};
    for (int i = 0; i < num_syms; i++) lookup[table[i].symbol] = &table[i];

    size_t total_bits = 0;
    for (long i = 0; i < file_size; i++) total_bits += lookup[data[i]]->code_len;
    >code_len;

    BitBuffer buf;
    buf_init(&buf, (total_bits + 7) / 8 + 16);
    size_t bit_offset = 0;
    for (long i = 0; i < file_size; i++) {
        CodeEntry *e = lookup[data[i]];

```

```

        buf_ensure(&buf, (bit_offset + e->code_len + 7) / 8);
        setbits(buf.data, bit_offset, e->code_val, e->code_len);
        bit_offset += e->code_len;
    }
}

```

```

FILE *fout = fopen(out_path, "wb");
if (!fout) { perror("Output file"); buf_free(&buf); free(data); return; }

```

```

uint32_t n = num_syms; //колво уник символов
uint64_t orig_len = file_size; //исходный размер файла
fwrite(&n, sizeof(n), 1, fout); //заголовок (4байта)
fwrite(&orig_len, sizeof(orig_len), 1, fout); //8байт

```

```

    fwrite(table, sizeof(CodeEntry), num_syms, fout); //таблица
    CodeEntry *num_syms
    fwrite(buf.data, 1, (bit_offset + 7) / 8, fout); //бит поток
    fclose(fout);

```

```

//размеры компонентов
size_t bitstream_bytes = (bit_offset + 7) / 8;
if (bitstream_bytes == 0) bitstream_bytes = 1; //защита от 0
size_t header_bytes = sizeof(n) + sizeof(orig_len) + (size_t)num_syms *
sizeof(CodeEntry);
size_t total_bytes = header_bytes + bitstream_bytes;

```

```

//кэффы
double k_raw = (double)file_size / bitstream_bytes; //теоретический
(только данные)
double k_total = (double)file_size / total_bytes; //фактический (файл
целиком)

```

```

printf("Сжатие завершено:\n");
printf("\tИсходный файл: %ld байт\n", file_size);
printf("\tБитовый поток (без заголовка): %zu байт (k=%.2f)\n",
bitstream_bytes, k_raw);
printf("\tПолный архив (с заголовком): %zu байт (k=%.2f)\n",
total_bytes, k_total);

```

```

    buf_free(&buf);
    free(data);
}

```

```

void decompress(const char *in_path, const char *out_path) { //распаковка
    FILE *fin = fopen(in_path, "rb");
    if (!fin) { perror("Input file"); return; }

```

```

    uint32_t num_syms;
    uint64_t orig_len;
    if (fread(&num_syms, sizeof(num_syms), 1, fin) != 1) { fprintf(stderr,
"Чтение заголовка\n"); fclose(fin); return; }
    if (fread(&orig_len, sizeof(orig_len), 1, fin) != 1) { fprintf(stderr,
"Чтение длины\n"); fclose(fin); return; }

```

```

    CodeEntry *table = malloc(num_syms * sizeof(CodeEntry));
    fread(table, sizeof(CodeEntry), num_syms, fin);

```

```

    long data_start = ftell(fin);
    fseek(fin, 0, SEEK_END);
    long total_size = ftell(fin);
    long bitstream_size = total_size - data_start;
    fseek(fin, data_start, SEEK_SET);

```

```

    uint8_t *bitstream = malloc(bitstream_size);
    fread(bitstream, 1, bitstream_size, fin);
    fclose(fin);

```

```

    FILE *fout = fopen(out_path, "wb");
    if (!fout) { perror("Output file"); free(bitstream); free(table); return; }

```

```

    uint32_t acc = 0;
    int acc_len = 0;
    size_t bit_offset = 0;
    size_t total_bits = bitstream_size * 8;
    uint64_t decoded = 0;

```

```

    while (decoded < orig_len && bit_offset < total_bits) {
        acc = (acc << 1) | getbits(bitstream, bit_offset, 1);
        acc_len++;
        bit_offset++;
    }

```

```

    for (uint32_t i = 0; i < num_syms; i++) {
        if (table[i].code_len == acc_len && table[i].code_val == acc) {
            fputc(table[i].symbol, fout);
            decoded++;
        }
    }

```

```

        acc = 0;
        acc_len = 0;
        break;
    }
}

fclose(fout);
free(bitstream);
free(table);
printf("Распаковка завершена: %lu символов восстановлено\n",
(unsigned long)decoded);
}

```

main.c:

```

#include "archive.h"
#include <stdio.h>
#include <string.h>
#include <getopt.h>

int main(int argc, char *argv[]) {
    int c_flag = 0, d_flag = 0;
    char *out_file = NULL;
    int opt;

    while ((opt = getopt(argc, argv, "cdo:")) != -1) { //с и d флаги без
аргументов, о: с арг
        switch (opt) { //выборка
            case 'c': c_flag = 1; break;
            case 'd': d_flag = 1; break;
            case 'o': out_file = optarg; break;
            default:
                fprintf(stderr, "Использование: %s {-c|-d} -o output_file
input_file\n", argv[0]);
                return 1;
            }
        }

    //проверка на вход файл (optindex)
    if (optind >= argc) {
        fprintf(stderr, "Ошибка: не указан входной файл.\n");
        return 1;
    }
    const char *in_file = argv[optind];

    //валидность флагов
    if (!out_file) {
        fprintf(stderr, "Ошибка: необходимо указать выходной файл
опцией -o.\n");
        return 1;
    }

    if (!c_flag && !d_flag) {
        fprintf(stderr, "Ошибка: укажите -с или -d.\n");
        return 1;
    }

    if (c_flag && d_flag) {
        fprintf(stderr, "Ошибка: нельзя одновременно -с и -d.\n");
        return 1;
    }

    //выполнение
    if (c_flag) {
        compress(in_file, out_file);
    } else {
        decompress(in_file, out_file);
    }

    return 0;
}

```